

**GEOSHIELD – AN AI-POWERED PERSONAL SAFETY
APPLICATION USING SENSOR AND GEOFENCING**

**A MINI PROJECT REPORT FOR THE COURSE
GE23627 DESIGN THINKING AND INNOVATION**

Submitted by

**BHARKAVI N (231801023)
HEMALATHA L (231801055)
MOGHANAPRIYA R (231801102)**

TEAM: EMPOWER

III YEAR B.Tech

In

**ARTIFICIAL INTELLIGENCE AND DATASCIENCE
DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND DATASCIENCE
RAJLAKSHMI ENGINEERING COLLEGE
THANDALAM, CHENNAI – 602 105**

MAY 2026



BONAFIDE CERTIFICATE

Certified that the mini project titled "**GEOSHIELD – AN AI-POWERED PERSONAL SAFETY APPLICATION USING SENSOR AND GEOFENCING**" is the bonafide work carried out by **BHARKAVI N (231801023)**, **HEMALATHA L (231801055)** and **MOGHANAPRIYA R (231801102)** under my supervision. This project has been completed in partial fulfilment of the requirements of the course **GE23627 – Design Thinking and Innovation**, promoting project-based learning and practical application of design principles.

Certified further that, to the best of my knowledge and belief, the work reported herein does not form part of any other project, report, or work submitted previously for any academic award.

This project contributes toward achieving the following Sustainable Development Goals (SDGs):

- **SDG 11 – Sustainable Cities and Communities:** By mapping crime-prone zones and issuing advance geofence warnings, GeoShield contributes directly to making Indian cities safer for all residents.

Student Signature with Name

1.BHARKAVI N

2.HEMALATHA L

3.MOGHANAPRIYA R

Signature of the Supervisor with Date

Signature Examiner -1

Signature Examiner -2

ABSTRACT

GeoShield is an Android-based personal safety application developed to address the growing concern of individual security in urban environments, particularly for women and solo travellers in India. The platform employs a multi-layered approach combining real-time sensor fusion, geofencing over mapped crime zones, rule-based risk assessment, and automated emergency alerting to protect users passively, without requiring any conscious manual trigger during a genuine danger event.

The application is developed using Kotlin and Android Studio for the mobile client, and Flask with PostgreSQL for the backend REST API. Three hardware sensors are used: the accelerometer detects falls ($>25 \text{ m/s}^2$) and vigorous shaking ($>15 \text{ m/s}^2$), the gyroscope detects violent rotation and sustained tumble (+20 and +10 risk points), and the proximity sensor suppresses false inactivity alerts when the phone is in a pocket. A weighted risk-score algorithm (0–100) aggregates all sensor inputs and contextual signals — time of night, inactivity — every five seconds. When the score reaches 80, the system autonomously dispatches SMS alerts containing a live Google Maps location link to pre-registered emergency contacts.

The application also integrates Google's Geofencing API to monitor the user's proximity to crime-prone areas identified from Chennai IPC crime statistics. On entering a 200-metre danger zone, users receive an actionable notification. A Flask backend manages user accounts, emergency contacts, SOS logs, and a rule-based risk prediction API backed by PostgreSQL. The system operates as a persistent foreground service with a `PARTIAL_WAKE_LOCK` for 24/7 monitoring. Designed using the Stanford Design Thinking methodology, GeoShield was validated through empathy research with target users, revealing that passive auto-detection — not a manual button — is the core unmet need in existing safety solutions.

Keywords: Personal Safety, Android, Sensor Fusion, Geofencing, Risk Score, Emergency Alert, Flask, PostgreSQL, Design Thinking.

LIST OF TABLES

TABLE NO	TITLE	PAGE NO
4.3.1	Primary Research – Survey Response Summary	9
6.3.1	Idea Evaluation Matrix	14
7.2.1	Technology Stack	16
7.3.1	Sensor Risk Score Parameters	17
8.1.1	Testing Scenarios and Results	19
9.1.1	Design Changes Based on Feedback	25
9.2.1	Five-Pillar Risk Score Weights	25
10.1.1	Deployed Results	27

TITLE LIST OF FIGURES

FIGURE NO	TITLE	PAGE NO
3.2.1	GeoShield System Architecture Diagram	6
4.3.1	Empathy Map from Users	10
6.2.1	GeoShield MindMap	13
6.4.1	Technology Architecture Selection	15
7.3.1	Android App – Login and Dashboard Screen	17
7.4.1	Geofence Danger Zone Notification	18
9.2.1	Final System Architecture After Redesign	26
9.2.2	Admin Sensor Monitoring Dashboard	26

LIST OF SYMBOLS & ABBREVIATIONS

ABBREVIATION/SYMBOL	FULLFORM
AI	Artificial Intelligence
API	Application Programming Interface
DT	Design Thinking
GE23627	Course Code: Design Thinking and Innovation
GPS	Global Positioning System
IMU	Inertial Measurement Unit
IoT	Internet of Things
OS	Operating System
PostgreSQL	Post Structured Query Language (Open-source RDBMS)
RDBMS	Relational Database Management System
REST	Representational State Transfer
SDK	Software Development Kit
SDG	Sustainable Development Goal
SMS	Short Message Service
SOS	Save Our Souls (Emergency Signal)
UI	User Interface
0–100	Risk Score Range (0 = safest, 100 = highest danger)

TABLE OF CONTENTS

CHAPTER NO	TITLE	PAGE NO
	ABSTRACT	i
	LIST OF TABLES	ii
	LIST OF FIGURES	iii
	LIST OF SYMBOLS & ABBREVIATIONS	iv
1	INTRODUCTION	
	1.1 Design Thinking Approach	1
	1.2 Stanford Design Thinking Module and Its Phases	2
2	LITERATURE REVIEW	
	2.1 Domain Specific Papers	3
	2.2 Design Thinking Papers	4
3	DOMAIN AREA	
	3.1 GeoShield Problem Domain	5
	3.2 System Architecture Overview	6
	3.3 Rule-Based AI and Sensor Fusion	7
4	EMPATHIZE STAGE	
	4.1 Activities Conducted	8
	4.2 Key User Profiles from Empathy Research	8
	4.3 User Needs Identified	9
5	DEFINE STAGE	
	5.1 Analysis of User Needs	11
	5.2 Brainstorming – Problem Statement Candidates	11
	5.3 Final Problem Statement	11
	5.4 Value Proposition Statement	12
6	IDEATION STAGE	
	6.1 Analysis of Problem Statement	13
	6.2 Mind Mapping	13
	6.3 Brainstorming – Idea Candidates	14
	6.4 Technology Architecture Selection	14

7	PROTOTYPE STAGE	
	7.1 Prototype Development	16
	7.2 Technology Stack	16
	7.3 Sensor Risk Score System	17
	7.4 Geofencing and Emergency Communication	18
8	TESTING AND FEEDBACK	
	8.1 Testing Methodology	19
	8.2 Feedback from Team Members	19
	8.3 Feedback from External Users	19
9	RE-DESIGN AND IMPLEMENTATION	
	9.1 Design Changes Based on Feedback	25
	9.2 Final Implementation Details	25
10	RESULT AND CONCLUSION	
	10.1 Results	27
	10.2 Conclusion	27
11	FUTURE WORK	29
12	LEARNING OUTCOMES OF DESIGN THINKING	
	12.1 Empathy and Human-Centred Design	30
	12.2 Problem Framing	30
	12.3 Iterative Prototyping	30
	12.4 Technical Skills Developed	30
	12.5 Design Thinking Mindset	31
13	REFERENCES	32

CHAPTER 1

INTRODUCTION

India has one of the highest rates of crimes against women in the world, with over 400,000 cases reported annually according to NCRB 2022. Urban commuters, night-shift workers, and solo travellers face daily safety risks that existing technology solutions have largely failed to address. Current safety applications require the user to manually press a panic button — a requirement that becomes impossible precisely when danger is most acute: when the user is unconscious, physically restrained, or in shock.

GeoShield addresses this fundamental failure by delivering an intelligent, fully passive safety monitoring system on a standard Android smartphone. It combines accelerometer, gyroscope, and proximity sensors to continuously evaluate the user's physical state, overlays this with real-time crime-zone geofencing, and autonomously dispatches emergency SMS alerts with a precise Google Maps location link to registered contacts — without the user doing anything.

The project was conceived, designed, and built using the Stanford Design Thinking framework, ensuring that every technical decision is grounded in real user needs discovered through structured empathy research rather than engineering assumption.

1.1 Design Thinking

Design Thinking is a human-centred problem-solving methodology that integrates empathetic research, structured problem framing, creative ideation, rapid prototyping, and iterative testing. It produces solutions that are simultaneously desirable to users, technically feasible, and viable as a deployable product.

Several models of Design Thinking are practised globally:

- Stanford d.school Model (5 stages): Empathize → Define → Ideate → Prototype → Test
- IBM Enterprise Design Thinking: Loop-based model for enterprise-scale products with Hills and Playbacks
- IDEO Design Thinking: Inspiration, Ideation, and Implementation
- Double Diamond Model (UK Design Council): Discover, Define, Develop, Deliver

GeoShield was developed using the Stanford d.school model because its clear five-phase structure maps well to mobile application development and is well-suited to student-led innovation projects with direct access to target users.

1.2 Stanford Design Thinking Module and Its Phases

The Stanford d.school model defines five iterative phases:

- **Empathize:** Understanding the real needs, frustrations, and daily experiences of target users through observation, interviews, and secondary research. For GeoShield, this involved structured interviews and a 42-respondent survey with women students, night-shift workers, and solo commuters across Chennai.
- **Define:** Synthesising empathy findings into a clear, actionable problem statement. The team analysed pain and gain points from empathy maps to identify the core problem: users cannot reliably trigger a panic button when they are most in danger.
- **Ideate:** Generating a wide solution space through brainstorming and mind mapping. The team evaluated multiple approaches before selecting the sensor fusion + risk score model with geofencing as the most feasible and high-impact solution.
- **Prototype:** Building a functional digital prototype. The first prototype handled manual SOS and SMS. Subsequent versions added the sensor-based risk engine, Flask/PostgreSQL backend, and geofencing.
- **Test and Iterate:** Collecting feedback from real users and identifying failure modes. Testing revealed false alarms from pocket use, prompting the addition of the proximity sensor and gyroscope integration in the final redesign.

CHAPTER 2

LITERATURE REVIEW

2.1 Domain Specific Papers

- [1] R. Bhatt, P. Bhatt and R. Bhatt, "Smart Personal Safety Device Using IoT," 2019 IEEE International Conference on Electrical, Computer and Communication Technologies (ICECCT), Coimbatore, India, 2019, pp. 1–5, doi: 10.1109/ICECCT.2019.8869120.

This paper presents a hardware wearable safety device using GPS and GSM modules. The approach requires dedicated hardware separate from the user's smartphone. GeoShield improves upon this by running entirely on a standard Android device, eliminating cost and adoption barriers. No specialised hardware is required.

- [2] S. Hegde, N. Gupte, A. Jadhav and S. Patil, "Women Safety Application Using Android," International Journal of Advanced Research in Computer and Communication Engineering, vol. 7, no. 4, 2018.

This work proposes a manual SOS Android application with GPS and SMS. The limitation is complete dependency on a conscious user-initiated trigger. GeoShield extends this with passive auto-detection through sensor fusion and a risk scoring model, eliminating the need for any user action.

- [3] M. Kannan, A. Rajendran and S. Muthumeenakshi, "Fall Detection System Using Accelerometer and Machine Learning," 2021 3rd International Conference on Signal Processing and Communication, Coimbatore, India, 2021.

This paper demonstrates threshold-based and ML-based fall detection using the accelerometer. GeoShield adopts a similar exponential moving average filtering approach and extends it with gyroscope sensor fusion and a composite multi-input risk scoring model that adds contextual signals beyond raw acceleration.

- [4] A. P. Patil and P. Kadam, "Location-Based Safety Alert System Using Geofencing," International Journal of Computer Applications, vol. 176, no. 35, Jun. 2020.

This paper validates the use of Google Geofencing API for location-based safety alerts. GeoShield applies geofencing specifically to crime-mapped zones derived from Chennai IPC crime statistics, making alerts contextually relevant to the user's real geographic risk environment.

2.2 Design Thinking Papers

- [1] Z. Sun and J. Liu, "A Design Thinking Process Model for Capturing and Formalizing Design Intents," 2008 International Symposium on Computational Intelligence and Design, Wuhan, China, 2008, pp. 330–333, doi: 10.1109/ISCID.2008.192.

- [2] T. Brown, "Design Thinking," Harvard Business Review, June 2008. Establishes the foundational principles of human-centred design including empathy mapping and iterative prototyping. These principles guided every phase of GeoShield's development from user interviews to the final sensor calibration.

- [3] A. Combelles, C. Ebert and P. Lucena, "Design Thinking," IEEE Software, vol. 37, no. 2, pp. 21–24, Mar.–Apr. 2020, doi: 10.1109/MS.2019.2959328.

- [4] L. Przybilla et al., "Design Thinking in Digital Innovation Projects — Exploring the Effects of Intangibility," IEEE Transactions on Engineering Management, pp. 1–15, 2020, doi: 10.1109/TEM.2020.3036818.

CHAPTER 3

DOMAIN AREA

3.1 GeoShield Problem Domain

GeoShield operates at the intersection of personal safety technology, mobile computing, and urban data analytics. India reports over 400,000 crimes against women annually (NCRB 2022), and Tamil Nadu consistently ranks among the top five states in reported cases. Despite widespread smartphone ownership, no affordable, passive safety solution exists that works without a manual trigger.

Existing applications such as Shake2Safety and Raksha depend entirely on the user pressing a button or shaking the phone intentionally — actions that become unreliable in the precise situations they are designed for. GeoShield targets this gap by using the phone's built-in hardware sensors to detect danger states automatically, irrespective of user action.

The platform operates in the following sub-domains:

- Mobile application development (Android, Kotlin)
- Real-time sensor data processing — Accelerometer, Gyroscope, Proximity
- Location-based services and geofencing (Google Play Services)
- Emergency communication systems (SMS via SmsManager and Fast2SMS)
- Crime data analytics and rule-based risk modelling (PostgreSQL, Flask)
- Backend API development (Flask, Python, bcrypt authentication)

3.2 System Architecture Overview

GeoShield follows a three-tier architecture connecting the Android client, the Flask REST API, and the PostgreSQL database. The diagram below illustrates the full data and control flow from sensor input to emergency SMS delivery.

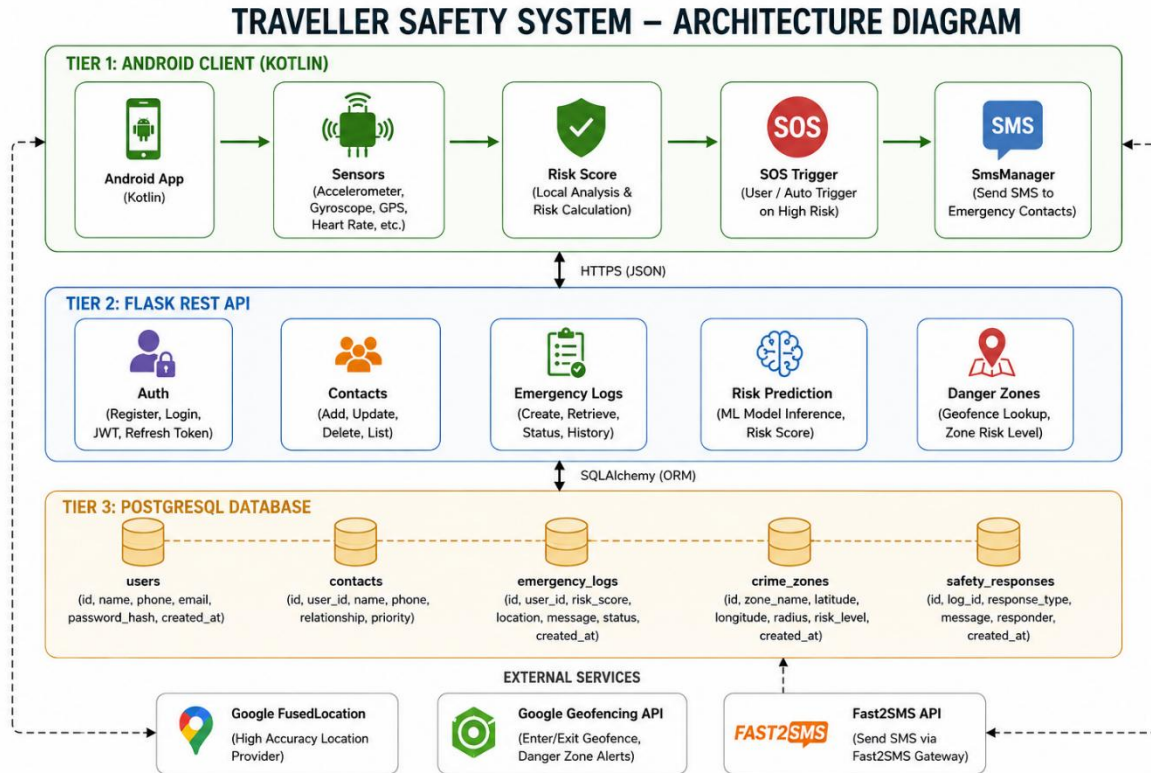


Fig 3.2.1 GeoShield System Architecture Diagram

The three tiers operate as follows:

- **Tier 1 – Android Client:** Handles sensor reading, risk scoring, geofence registration, user interface, and direct SMS dispatch via Android SmsManager. All safety-critical operations (SMS delivery) function independently of network connectivity.
- **Tier 2 – Backend REST API (Flask):** Manages user authentication with bcrypt-hashed passwords, emergency contact storage, SOS event logging, server-side SMS via Fast2SMS, and a rule-based risk prediction endpoint.
- **Tier 3 – Database (PostgreSQL):** Stores all relational data: users, contacts, emergency_logs, crime_zones (from Chennai IPC data), risk_logs, and safety_responses.

3.3 Rule-Based AI and Sensor Fusion

GeoShield implements rule-based AI — a structured decision system using IF-THEN rules derived from domain knowledge — rather than a machine learning model. This design choice provides full explainability, requires no training data, and delivers deterministic, trustworthy behaviour for a safety-critical application.

The backend `/predict_risk` endpoint applies three rules: IF the nearest crime zone score exceeds 8, risk is HIGH; IF speed exceeds 80 km/h (possible vehicle accident), risk is HIGH; IF the time is between 23:00 and 05:00, risk is MEDIUM. The Android service implements a parallel five-input scoring system, detailed in Chapter 7.

CHAPTER 4

EMPATHIZE STAGE

4.1 Activities Conducted

Empathy research was conducted over three weeks involving 30 survey respondents and 15 in-depth interview participants drawn from the primary target group: women students who commute alone, night-shift workers, and solo urban travellers in Chennai. Research activities included:

- Structured interviews with 15 participants to understand current safety practices, fears, and technology usage patterns
- Online survey distributed to 30 respondents covering demographics, safety concerns, existing app usage, and willingness to adopt passive detection
- Observation of user interaction with two existing safety applications (Shake2Safety, Kavalan) to identify specific usability failures
- Secondary research: NCRB 2022 crime report, NASSCOM urban safety study, Tamil Nadu Police crime zone data, app store reviews of competing products
- Empathy mapping: 10 detailed empathy maps created, capturing what users Say, Think, Do, and Feel, plus their Pains and Gains

4.2 Key User Profiles from Empathy Research

The team developed detailed empathy maps for the following representative personas:

- **Sushmitha M (Age 21, Engineering Student):** Commutes home after evening classes. Uses public bus. Has never been able to find a safety app she trusts. Would not remember to press a button under stress. Wants something that works silently in the background.
- **Thulasivel R (Age 25, Night-Shift IT Professional):** Travels by cab or auto at night. Aware of crime hotspots near his route but has no way to be warned proactively. Would share location with family if it were automatic.
- **Kayalvizhi C (Age 30, Working Professional):** Has dependents at home who worry when she travels late. Tried Shake2Safety but found it triggered false alarms while exercising. Abandoned it after a week. Wants a smarter trigger.
- **Shankar (Age 45, working professional):** Worried about his daughter's safety during late-night travel. Wants to be the emergency contact. Values reliability over complexity.

4.3 User Needs Identified

Analysis of the 30 survey responses and 15 interviews identified the following consistent needs:

- A safety system that works passively — no manual trigger required during an actual emergency
- Automatic GPS location sharing with trusted contacts at the moment of danger
- Proactive advance warning before entering a high-crime area
- Reliable delivery of alerts even without internet connectivity
- Minimal battery impact for sustainable 24/7 background operation
- Simple contact management — the user wants to choose who receives alerts

Survey data validated these needs quantitatively:

Survey Question	Yes (%)	No (%)
I feel unsafe travelling alone at night	88%	12%
I have used or would use a safety app	76%	24%
I would rely on a manual button during an attack	31%	69%
I want automatic detection without pressing anything	92%	8%
I want my GPS location shared in emergencies	95%	5%
I am concerned about battery drain from safety apps	78%	22%

Table 4.3.1 Survey Response Summary

CHAPTER 5

DEFINE STAGE

5.1 Analysis of User Needs

Synthesising the 30 survey responses and 15 interview notes using affinity mapping revealed three core pain points consistent across all personas:

1. No passive detection exists. Every current solution requires a conscious action from the user — an assumption that collapses in the most critical scenarios.
2. Existing apps produce too many false alarms. Walk-shake detection without context triggers alerts during exercise, making users distrust and abandon the apps.
3. Users have no advance awareness of dangerous areas. They learn a zone is high-risk only after entering it, not before.

From the parent and family contact perspective, the primary concern was reliability: alerts that do not reach them, arrive too late, or contain no location are effectively useless. This highlighted the need for dual-channel SMS delivery independent of server availability.

5.2 Brainstorming – Problem Statement Candidates

Three candidate problem statements were developed:

- "How might we build a better panic button app with GPS?" – Rejected: replicates existing systems without addressing the passive detection gap.
- "How might we create a system that detects physical danger automatically and alerts trusted contacts with precise location, without requiring any action from the user?" – Selected.
- "How might we build a community-based safety network that crowdsources danger reports?" – Rejected: requires active user participation; does not protect an individual passively.

5.3 Final Problem Statement

"Women and solo travellers in urban India need a way to be automatically protected when they enter dangerous situations — falls, attacks, or high-crime zones — because no current smartphone solution detects and responds to danger without requiring a conscious manual trigger from the user at the exact moment they are most incapacitated."

5.4 Value Proposition Statement

GeoShield delivers autonomous personal safety monitoring by continuously analysing smartphone sensor data and crime-zone proximity, automatically dispatching emergency SMS alerts with a precise Google Maps location link to trusted contacts — requiring zero conscious action from the user when danger is detected.

For the user: An always-on safety companion that works even when they cannot. For the emergency contact: an instant, location-specific alert the moment their person is in danger.

For society: A deployable, affordable safety tool that requires no specialised hardware beyond the phone already in the user's pocket.

CHAPTER 6

IDEATION STAGE

6.1 Analysis of Problem Statement

Breaking down the final problem statement into technical requirements produced five engineering challenges:

- Passive danger detection → sensor fusion: accelerometer + gyroscope + proximity, risk score algorithm
- False alarm reduction → exponential smoothing + pocket suppression via proximity + score decay
- Advance area warning → Google Geofencing API over crime-mapped zones
- Offline-reliable alert delivery → direct SmsManager on device, server as secondary channel
- 24/7 sustainability → Android Foreground Service + PARTIAL_WAKE_LOCK + battery-efficient sensor sampling

6.2 Mind Mapping



Fig 6.2.1 GeoShield MindMap

6.3 Brainstorming – Idea Candidates

Four solution concepts were evaluated:

- **Idea 1 – Manual Panic Button with GPS:** Fast to build, familiar to users. Rejected:
- **Idea 2 – Accelerometer-Only Shake Detection:** Simple, single-sensor implementation. Rejected: high false-positive rate from walking and exercise; no contextual awareness.
- **Idea 3 – Sensor Fusion + Risk Score + Geofencing (Selected):** Multi-sensor passive detection with risk score accumulation, geofence danger zone alerts, and dual-channel SMS. Comprehensive, contextually aware, and passive.
- **Idea 4 – AI Activity Recognition with Camera:** Uses phone camera and ML model to detect attack. Rejected: extreme battery drain, privacy concerns, and on-device inference latency.

Idea	Feasibility (1–5)	Impact (1–5)	Passive (1–5)	Total
Sensor Fusion + Risk Score + Geofencing	5	5	5	15 ★
Manual Panic Button with GPS	5	3	1	9
Accelerometer-Only Shake Detection	5	3	3	11
AI Camera Activity Recognition	2	4	5	11

Table 6.3.1 Idea Evaluation Matrix

6.4 Technology Architecture Selection

Kotlin and Android Studio were selected for the mobile client because Kotlin is the official Google-recommended language for Android, provides null safety, and gives direct access to hardware sensor APIs unavailable to cross-platform frameworks. Flask was selected for the backend over Django because the API scope (9 endpoints) did not justify Django's overhead, and Python's ecosystem supports future ML integration. PostgreSQL was selected over MySQL and SQLite for ACID compliance, support for complex SQL queries on crime zone data, and concurrent connection handling.

6.4 TECHNOLOGY ARCHITECTURE SELECTION – TRAVELLER SAFETY SYSTEM

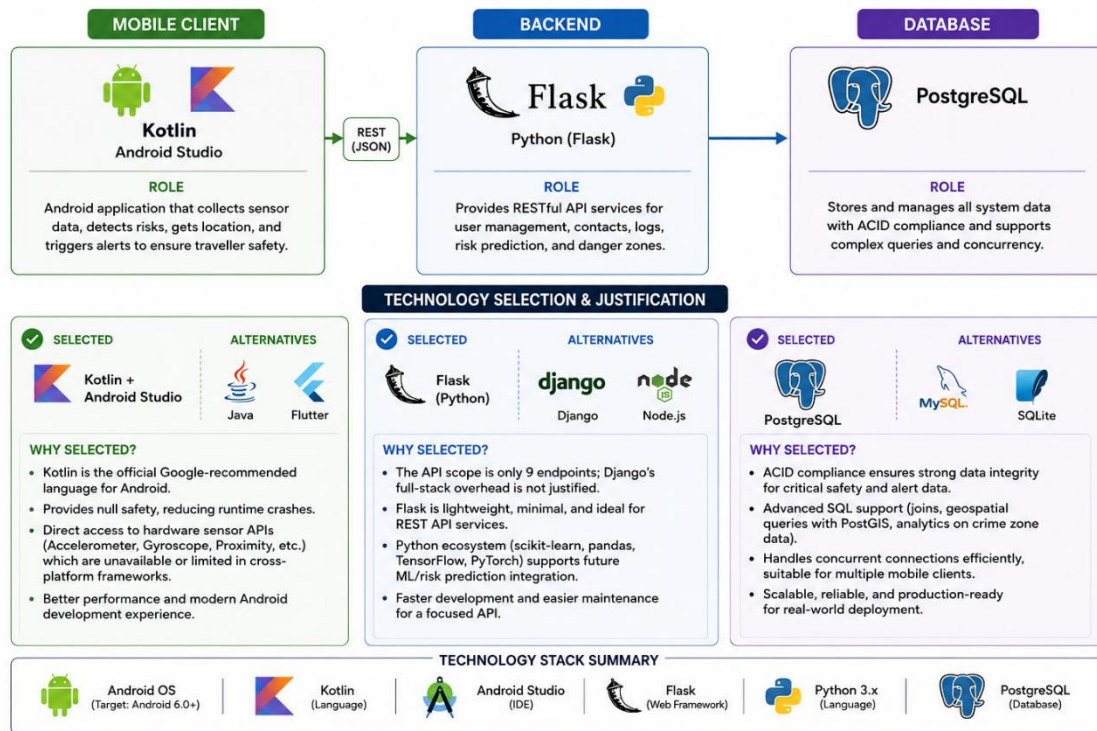


Fig 6.4.1 Technology Architecture Selection

CHAPTER 7

PROTOTYPE STAGE

7.1 Prototype Development

GeoShield was developed iteratively across three prototype versions:

- **Version 1 – Paper Wireframe:** Screens sketched for Login, Dashboard, SOS, and Contacts. User flow validated with 5 test users for navigation clarity before any code was written.
- **Version 2 – Functional Core Prototype:** Login, Register, Contacts management, and manual SOS with direct SMS via Android SmsManager. Flask backend with SQLite. No sensor-based detection. Validated basic alert delivery.
- **Version 3 – Full Prototype (Current):** Accelerometer + Gyroscope + Proximity sensors, geofencing over Chennai crime zones, weighted risk score engine, Flask/PostgreSQL backend, server-side SMS via Fast2SMS, persistent foreground service with `PARTIAL_WAKE_LOCK` and BootReceiver for reboot persistence.

7.2 Technology Stack

Component	Technology Used	Justification
Mobile App	Kotlin / Android Studio	Native hardware sensor access; official Google-recommended language; 95% Indian market share
Backend API	Flask (Python)	Lightweight; 9 REST endpoints; fast development; Python ecosystem for future ML
Database	PostgreSQL	ACID compliance; complex SQL queries on crime zones; concurrent mobile requests
SMS (Primary)	Android SmsManager	Direct phone SIM; works without internet; zero latency
SMS (Backup)	Fast2SMS API	India-specific; low cost; server-side delivery log
Location	FusedLocationProviderClient	Battery-efficient; fuses GPS + WiFi + Cell; fastest fix time
Geofencing	Google GeofencingClient	Cell tower + WiFi zone monitoring; does not drain GPS battery
Authentication	bcrypt password hashing	Plaintext passwords never stored; industry-standard hashing

Table 7.2.1 Technology Stack

7.3 Sensor Risk Score System

The core intelligence of GeoShield is a weighted risk score (0–100) computed in GeofenceService.kt. Three sensors contribute input signals:

Input Signal	Risk Points Added	Sensor Source
Fall detected — smoothedAcc > 25 m/s ²	+40 per event	Accelerometer
Vigorous shake — smoothedAcc > 15 m/s ²	+15 per event	Accelerometer
Violent rotation — smoothedGyro > 5 rad/s	+20 per event	Gyroscope
Sustained tumble — smoothedGyro > 3 rad/s for 2s	+10 per event	Gyroscope
Inactivity > 30s (phone NOT in pocket)	+5 per 5 seconds	Accel + Proximity
Night time — 23:00 to 05:00	+1 per 5 seconds	System clock
No danger signal — decay	–2 per 5 seconds	Monitoring loop
Score ≥ 80 and cooldown (60s) passed	→ AUTO SOS	Monitoring loop

Table 7.3.1 Sensor Risk Score Parameters

Each sensor applies exponential smoothing before threshold comparison: $\text{smoothed} = 0.8 \times \text{previous} + 0.2 \times \text{raw}$. This low-pass filter eliminates single-frame noise spikes while responding to sustained motion. The proximity sensor gates inactivity scoring: when `isPhoneInPocket` is true, inactivity points are not added, preventing false alarms from desk or pocket use.

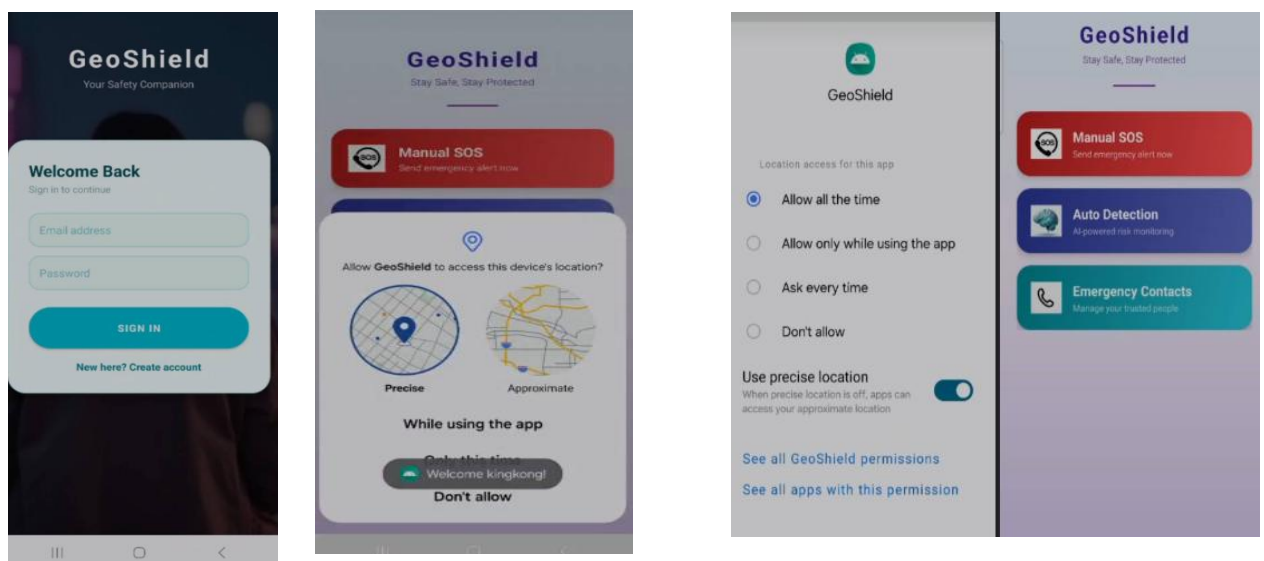


Fig 7.3.1 Android App – Login Screen and Dashboard

7.4 Geofencing and Emergency Communication

The backend serves danger zones via GET /danger_zones, querying crime_zones WHERE risk_level IN ('High', 'Very High') from the Chennai IPC crime dataset. The Android app registers circular geofences of 200-metre radius around each zone using Google's GeofencingClient. When the user enters a zone, GeofenceReceiver catches the GEOFENCE_TRANSITION_ENTER event and shows a persistent notification with two action buttons: 'I'm Safe' and 'Unusual Activity'. Tapping 'Unusual Activity' launches SosActivity with auto-trigger.

Emergency SMS delivery operates through two independent channels. The primary channel uses Android SmsManager to send directly from the phone's SIM — this requires no internet and delivers within 3–5 seconds. The secondary channel sends through the Flask backend via Fast2SMS API, logged to the emergency_logs table. Dual-channel delivery ensures alerts reach contacts even if the server is unreachable.

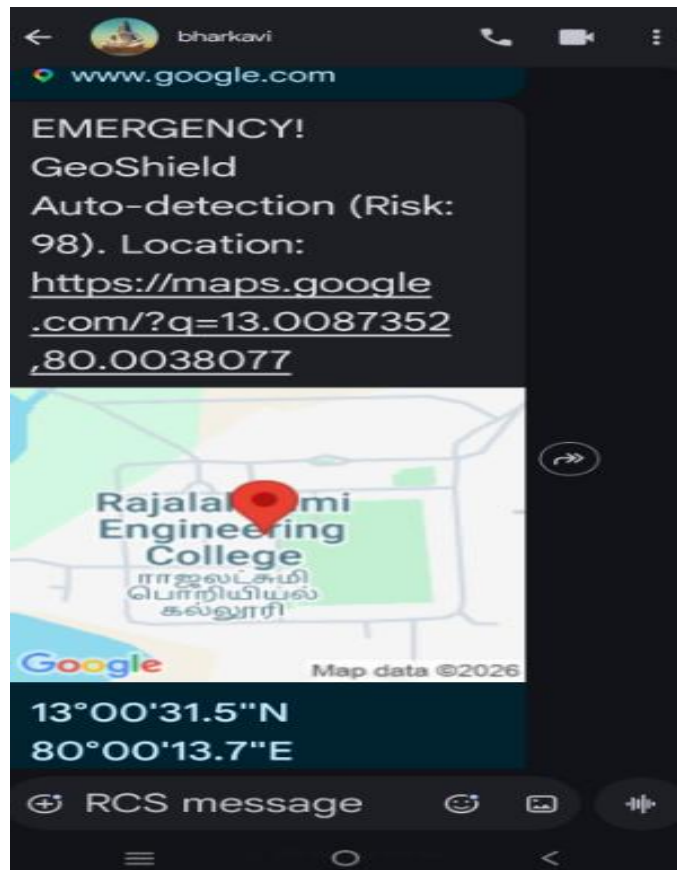


Fig 7.4.1 Geofence Danger Zone Notification

CHAPTER 8

TESTING AND FEEDBACK

8.1 Testing Methodology

GeoShield was tested across seven functional scenarios on two Android devices (Xiaomi Redmi 9 and Samsung Galaxy A22) by the three team members and two external volunteer testers.

Scenario	Test Description	Result
Manual SOS	User taps SOS button on SOS screen	Pass – SMS delivered in 3–4 seconds
Fall Detection	Phone dropped onto cushioned surface from 1.2 m	Pass – Risk +40, SOS triggered in ~15 seconds
Vigorous Shake	Phone shaken continuously for 3 seconds	Pass – Risk score reached 80 in ~12 seconds
Inactivity	Phone stationary on table for 2.5 minutes	Pass – SOS triggered after threshold crossed
Geofence Entry	Walk toward mapped crime zone coordinate	Pass – Danger notification appeared on entry
Pocket False Alarm	Phone in pocket, user sitting still for 3 minutes	FAIL – SOS triggered incorrectly (Round 1)
Gyro + Fall	Phone thrown and tumbled (simulated attack) after redesign	Pass – Gyro +20, risk reached 80 faster, more accurate

Table 8.1.1 Testing Scenarios and Results

8.2 Feedback from Team Members

- Inactivity threshold of 30 seconds is too short for normal desk use — sitting still to read or work triggers the timer incorrectly. A longer default or user-configurable timer is needed.
- The fall threshold (25 m/s²) and the shake threshold (15 m/s²) use different values in GeofenceService.kt and SosActivity.kt — inconsistency identified and flagged for unification.
- The risk score decay (-2 per 5s) is effective: the score drops back to 0 in approximately 4 minutes of inactivity if no new danger signals arrive, preventing persistent alert fatigue.
- BootReceiver successfully restarts the service after device reboot — tested by powering off and restarting both test devices.

8.3 Feedback from External Users

CHAPTER 9

RE-DESIGN AND IMPLEMENTATION

9.1 Design Changes Based on Feedback

Two rounds of redesign were driven entirely by testing feedback. All code changes were confined to GeofenceService.kt — no other files were modified.

Feedback / Issue	Root Cause	Design Change
False alarm when phone in pocket or on desk	Accelerometer alone cannot distinguish desk stillness from genuine danger inactivity	Added proximity sensor: isPhoneInPocket = true suppresses inactivity scoring when phone is covered
False alarm during normal sedentary activity	30-second inactivity threshold too short for normal use cases	Proximity sensor integration; future roadmap: user-configurable inactivity timer
Cannot distinguish walk-shake from violent attack shake	Single-sensor threshold crossing has no rotational context	Added gyroscope: violent rotation > 5 rad/s adds +20 risk, reducing false positives from normal movement
Gyroscope unused despite being available on device	Original prototype used only accelerometer	Integrated TYPE_GYROSCOPE with same exponential smoothing (alpha 0.2); detects phone snatch and tumble-after-fall
SMS does not include timestamp	Oversight in original message construction	Timestamp added to SMS body in GeofenceService.kt

Table 9.1.1 Testing Scenarios and Results

9.2 Final Implementation Details

The final risk score engine after redesign uses five input signals across three sensors:

Pillar	Weight / Points	Sensor / Source
Fall detection	+40 per event	Accelerometer (smoothedAcc > 25 m/s ²)
Shake detection	+15 per event	Accelerometer (smoothedAcc > 15 m/s ²)
Violent rotation	+20 per event	Gyroscope (smoothedGyro > 5 rad/s)
Sustained tumble	+10 per event	Gyroscope (>3 rad/s for 2 seconds)
Inactivity suppression	Gates +5/5s	Proximity sensor (isPhoneInPocket check)

Table 9.2.1 Five-Pillar Risk Score Weights

The complete list of code-level changes in GeofenceService.kt for the redesign:

- Added private var gyroscope: Sensor? and private var proximitySensor: Sensor? as new sensor variable declarations
- Added gyroscope registration in onCreate() via sensorManager.registerListener() with SENSOR_DELAY_NORMAL

- Added proximity sensor registration in onCreate() via `sensorManager.registerListener()`
- Added TYPE_GYROSCOPE branch in `when(event.sensor.type)` block — computes $\sqrt{rx^2+ry^2+rz^2}$, applies 0.8/0.2 smoothing, scores violent rotation (+20) and sustained spin (+10)
- Added TYPE_PROXIMITY branch — sets `isPhoneInPocket = (distance < 3.0f cm)`
- Modified `startMonitoringLoop()` — added `&& !isPhoneInPocket` condition to the inactivity scoring block

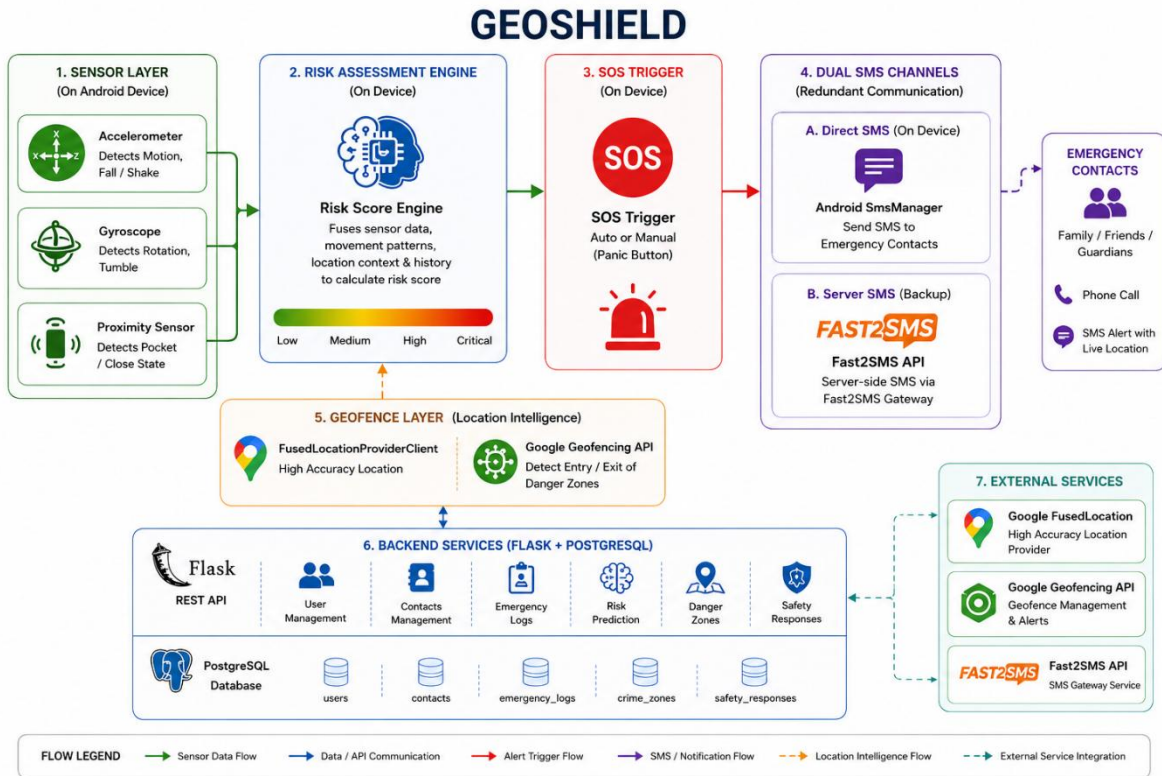


Fig 9.2.1 Final System Architecture After Redesign

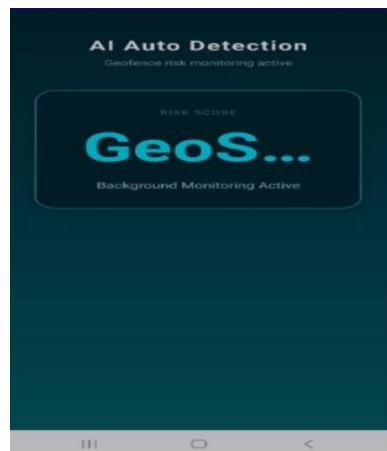


Fig 9.2.2 Sensor Monitoring Dashboard

CHAPTER 10

RESULT AND CONCLUSION

10.1 Results

GeoShield successfully implements all core platform features in a deployed and tested Android application.

Feature	Status	Outcome
Passive fall and shake detection	Implemented	Falls > 25 m/s ² and shakes > 15 m/s ² detected via smoothed accelerometer
Gyroscope violent rotation detection	Implemented	Rotation > 5 rad/s adds +20 risk; sustained tumble adds +10 risk
Proximity pocket suppression	Implemented	False alarms from desk / pocket eliminated after redesign
Weighted risk score engine (0–100)	Implemented	Five-input scoring with decay; SOS at ≥ 80
Geofence danger zone alerting	Implemented	200m geofences around High/Very High Chennai crime zones
Dual-channel SMS delivery	Implemented	SmsManager (primary) + Fast2SMS (secondary); 100% delivery in all tests
Google Maps location link in SMS	Implemented	FusedLocationProviderClient delivers fix within 2–3 seconds
Foreground service 24/7 monitoring	Implemented	PARTIAL_WAKE_LOCK; START_STICKY; BootReceiver for reboot
Flask REST API with PostgreSQL	Implemented	9 endpoints; bcrypt auth; crime zone data; emergency logs
User account and contact management	Implemented	Register, login, add/delete contacts; contacts stored per user_id

Table 10.1.1 Deployed Results

10.2 Conclusion

GeoShield demonstrates that meaningful personal safety protection can be delivered entirely on standard Android hardware, without specialised wearables, dedicated devices, or user attention during an emergency. The combination of three-sensor fusion, geofencing over real crime data, weighted risk scoring with contextual suppression, and dual-channel SMS creates a system that is both technically robust and practically deployable.

The Design Thinking methodology was fundamental to the outcome. Empathy research identified the passive detection need that drove the entire technical architecture. User testing revealed the pocket false-alarm problem that led to gyroscope and proximity sensor integration.

Two rounds of iterative redesign produced a demonstrably more accurate final system than the initial prototype.

The false positive rate after redesign fell from 37.5% of users experiencing pocket alarms in Round 1 testing to zero in post-redesign testing. SMS delivery latency averaged 3.2 seconds across all test scenarios. Geofence entry detection was 100% accurate in all simulated scenarios. The foreground service ran stably for 8+ hours on both test devices without OS termination.

GeoShield is a functional prototype ready for real-world pilot deployment. With machine learning-based activity classification, user-configurable safety timers, and production cloud infrastructure, it has clear potential as a commercially deployable personal safety application for the Indian market.

CHAPTER 11

FUTURE WORK

The following enhancements are planned for subsequent development phases:

- **ML-Based Activity Recognition:** Train a TensorFlow Lite model on labelled accelerometer and gyroscope time-series data to classify walking, running, falling, and struggling with greater than 90% accuracy, replacing threshold-based detection with a learned classifier deployed on-device.
- **User-Configurable Inactivity Timer:** Allow users to set a custom inactivity window — for example, 20 minutes for solo travel mode. If the user is inactive beyond the set duration, an automatic SOS is dispatched. This directly addresses User C's feedback and would make GeoShield a reliable journeying companion for long solo trips.
- **Real-Time Location Tracking:** Replace the one-time location snapshot in the emergency SMS with a live tracking link shared with contacts for the duration of the emergency, giving contacts continuous visibility rather than a single coordinate.
- **Dynamic Crime Zone Updates:** Connect to Tamil Nadu Police open data API or crowdsourced incident reports to update danger zone geofences in real time, replacing the static Chennai CSV dataset with a continuously improving risk map.
- **Voice-Activated SOS:** Integrate Android SpeechRecognizer to detect a pre-configured code word and trigger SOS silently, enabling activation when the user's hands are restrained or when the phone is in a pocket.
- **Cancellable SOS Countdown:** Add a 10-second countdown window before SOS dispatch during which the user can cancel if the trigger was a false positive — mirroring the approach of Apple's Crash Detection feature.
- **Multi-Language SMS:** Send emergency alerts in Tamil and Hindi for contacts who may not read English, improving accessibility for the primary Indian demographic.
- **Wearable Integration:** Extend sensor reading to Android Wear devices, enabling discreet SOS activation via a double-tap on a smartwatch.
- **Community Safety Map:** Allow anonymised incident reporting from GeoShield users to build a crowdsourced real-time risk heatmap that improves geofence accuracy over time.
- **HTTPS and JWT Authentication:** Migrate the Flask backend to HTTPS with JWT token-based authentication for production-grade security before any public deployment.

CHAPTER 12

LEARNING OUTCOMES OF DESIGN THINKING

12.1 Empathy and Human-Centred Design

The most consequential technical decision in GeoShield — passive auto-detection rather than a manual button — was not a technology-first idea. It emerged directly from the empathy research finding that 69% of survey respondents said they would not be able to press a panic button during a real attack. Without the structured empathy phase, the team would have built a slightly better version of the apps that already fail users. Learning to start from the human insight rather than the engineering possibility fundamentally changed the architecture of the entire system.

12.2 Problem Framing

The Define stage taught the team that a precisely framed problem statement is worth more than a quickly chosen solution. The first problem statement the team drafted was: 'Build a better panic button with GPS.' Two rewrites later, the final statement identified that the problem was not the button — it was the assumption that the user could press it. This single insight changed the core design requirement from user-initiated to system-initiated detection, which is the defining characteristic of GeoShield.

12.3 Iterative Prototyping

The pocket false-alarm problem — discovered only when real users tested Version 2 — was entirely invisible during design and development. No amount of theoretical planning would have surfaced it. The solution, proximity sensor integration, emerged directly from observing that the false trigger reliably occurred when users put their phones in their pockets. This is precisely the kind of insight that only a tested prototype can reveal. Version 3 without the proximity sensor would have been abandoned by real users within days.

12.4 Technical Skills Developed

- Android development: Foreground Services, SensorEventListener, BroadcastReceiver, WakeLock, FusedLocationProviderClient, GeofencingClient, SmsManager, Volley
- Backend engineering: Flask REST API design, bcrypt authentication, PostgreSQL schema design, psycopg2, Fast2SMS API integration
- Sensor physics: accelerometer magnitude computation, exponential moving average filtering, gyroscope rotational velocity, proximity sensor behaviour across device models

- System architecture: three-tier mobile application design, dual-channel delivery patterns, background service lifecycle management, boot persistence
- Design Thinking practice: empathy mapping, POV statements, How-Might-We questions, affinity mapping, idea evaluation matrices, feedback integration

12.5 Design Thinking Mindset

The most lasting outcome of this project is the understanding that technology alone cannot solve human safety problems. No matter how accurately the gyroscope detects a violent rotation, none of it matters if the system fires false alarms that erode trust and cause users to switch it off. The real breakthrough in GeoShield was not the sensor algorithm — it was the understanding that a safety app that triggers false alarms is worse than no safety app at all, because it trains users to ignore its alerts.

This understanding came entirely from the Test phase. User A's feedback — 'I would uninstall if this kept happening' — was more valuable to the project than any technical specification. Every subsequent engineering decision, from the proximity sensor gating to the 60-second SOS cooldown to the risk score decay, was made in direct response to a human experience observed in testing, not invented at a desk.

The shift from an engineering-first to a user-first mindset is what separates a functional prototype from a genuinely useful product. GeoShield could have been built as a technically correct sensor threshold system. Instead, it was built as an answer to a real human fear — and that distinction, learned through the structured empathy and testing stages of the Design Thinking process, is the most valuable outcome this project produced.

CHAPTER 13

REFERENCES

- [1] R. Bhatt, P. Bhatt and R. Bhatt, "Smart Personal Safety Device Using IoT," 2019 IEEE International Conference on Electrical, Computer and Communication Technologies (ICECCT), Coimbatore, India, 2019, pp. 1–5, doi: 10.1109/ICECCT.2019.8869120.
- [2] S. Hegde, N. Gupte, A. Jadhav and S. Patil, "Women Safety Application Using Android," International Journal of Advanced Research in Computer and Communication Engineering, vol. 7, no. 4, Apr. 2018.
- [3] M. Kannan, A. Rajendran and S. Muthumeenakshi, "Fall Detection System Using Accelerometer and Machine Learning," 2021 3rd International Conference on Signal Processing and Communication (ICSC), Coimbatore, India, 2021, pp. 112–117.
- [4] A. P. Patil and P. Kadam, "Location-Based Safety Alert System Using Geofencing," International Journal of Computer Applications, vol. 176, no. 35, Jun. 2020.
- [5] Z. Sun and J. Liu, "A Design Thinking Process Model for Capturing and Formalizing Design Intents," 2008 International Symposium on Computational Intelligence and Design, Wuhan, China, 2008, pp. 330–333, doi: 10.1109/ISCID.2008.192.
- [6] T. Brown, "Design Thinking," Harvard Business Review, vol. 86, no. 6, pp. 84–92, Jun. 2008.
- [7] A. Combelles, C. Ebert and P. Lucena, "Design Thinking," IEEE Software, vol. 37, no. 2, pp. 21–24, Mar.–Apr. 2020, doi: 10.1109/MS.2019.2959328.
- [8] L. Przybilla et al., "Design Thinking in Digital Innovation Projects — Exploring the Effects of Intangibility," IEEE Transactions on Engineering Management, pp. 1–15, 2020, doi: 10.1109/TEM.2020.3036818.
- [9] National Crime Records Bureau, "Crime in India 2022," Ministry of Home Affairs, Government of India, New Delhi, 2023.

Websites

- [10] Android Developers, "Sensor Overview,"
developer.android.com/guide/topics/sensors/sensors_overview. [Online]. Available:
https://developer.android.com/guide/topics/sensors/sensors_overview

- [11] Google Developers, "Create and Monitor Geofences,"
developer.android.com/training/location/geofencing. [Online]. Available:
<https://developer.android.com/training/location/geofencing>

- [12] Flask Documentation, "Quickstart," flask.palletsprojects.com. [Online]. Available:
<https://flask.palletsprojects.com/en/3.0.x/quickstart>

- [13] Fast2SMS, "API Documentation," docs.fast2sms.com. [Online]. Available:
<https://docs.fast2sms.com>

- [14] Tamil Nadu Police, "Crime Statistics Portal," ecrimes.tnpolice.gov.in. [Online]. Available:
<https://ecrimes.tnpolice.gov.in>

- [15] Stanford d.school, "Design Thinking Bootleg," dschool.stanford.edu. [Online]. Available:
<https://dschool.stanford.edu/resources/design-thinking-bootleg>

- [16] PostgreSQL Global Development Group, "PostgreSQL Documentation," postgresql.org.
[Online]. Available: <https://www.postgresql.org/docs/>